

UHack-a-Py

Bullet-Hell OOP template

0. Introductie

Hier zal jullie worden aangeleerd hoe je de **Bullet-Hell** template aanvult tot een werkend startpunt met behulp van **Object-Oriented Programming (OOP)**. Bullet-Hell is een genre van games waarbij de speler vijanden moet ontwijken of zelf afschiet. Enkele bekende voorbeelden zijn **Enter the Gungeon**, **Vampire Survivors** en **Brotato**, maar er zijn nog veel meer gelijkaardige games.

In deze OOP versie werken we met **klassen**. Elke klasse stelt een game-object voor en bevat de bijhorende eigenschappen (attributen) en gedrag (methoden). Zo hoef je niet langer aparte lijsten bij te houden voor posities en richtingen — elk object beheert die gegevens zelf.

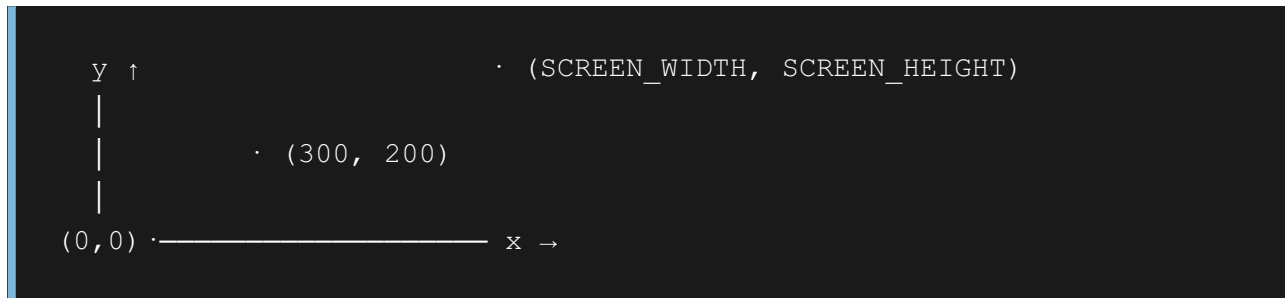


De template bevat drie objectklassen (**Player**, **Enemy** en **Bullet**) en een **Game**-klasse die alles coördineert. De meeste logica in de **Game**-klasse is al voor jullie ingevuld.

Om een goed begrip te krijgen van dit systeem en de templates raden we aan om deze cursus goed te volgen. Als er vragen zijn of iets niet duidelijk is, **aarzel niet om hulp te roepen!**

Het coördinatenstelsel

Voordat we beginnen is het belangrijk om te begrijpen hoe posities op het scherm werken. Het punt **(0, 0)** ligt **linksonder** op het scherm. De x-waarde loopt naar rechts en de y-waarde loopt **naar boven**.



Wil je iets helemaal rechtsboven tekenen? Dan gebruik je een x-waarde dicht bij **SCREEN_WIDTH** en een y-waarde dicht bij **SCREEN_HEIGHT**. Dit is een veelgemaakte fout, dus hou het goed in de gaten!

De gegeven code

De **`__init__`** methoden van de klassen zijn al volledig ingevuld. Hieronder vind je een overzicht van de attributen die automatisch worden aangemaakt wanneer een object wordt aangemaakt.

Klasse	Attribuut	Betekenis
Player	<code>self.position</code>	Positie van de speler als Vector (x, y)
	<code>self.size</code>	Breedte en hoogte in pixels (150)
	<code>self.speed</code>	Snelheid waarmee de speler beweegt (5.5)
	<code>self.direction</code>	Huidige bewegingsrichting als Vector (standaard (0,0))
Enemy	<code>self.position</code>	Positie van de vijand als Vector (x, y)
	<code>self.size</code>	Breedte en hoogte in pixels
	<code>self.speed</code>	Snelheid van de vijand
Bullet	<code>self.position</code>	Positie van de kogel als Vector (x, y)
	<code>self.size</code>	Breedte en hoogte in pixels
	<code>self.speed</code>	Snelheid van de kogel

`self.direction`

Richting van de kogel als Vector

Naast de attributen zijn er ook methoden in de `Game`-klasse al voor je voorzien:

Methode	Wat ze doet
<code>update_game_state()</code>	Beweegt alle objecten, spawnnt vijanden en detecteert botsingen
<code>detect_collisions()</code>	Berekent botsingen tussen kogels, vijanden en de speler
<code>spawn_enemies()</code>	Laat vijanden verschijnen aan de randen van het scherm
<code>remove_bullets_out_of_screen()</code>	Verwijdert kogels die buiten het scherm zijn
<code>show_game_over()</code>	Toont het game-over scherm en herstart het spel

De `move()` methode van `Player` is ook al ingevuld — die past de positie aan op basis van `self.direction` en `self.speed`. Jullie hoeven die niet aan te passen.

Frames

In een game wordt het scherm **niet één keer getekend**, maar **meerdere keren per seconde**. Elke keer dat het scherm opnieuw wordt getekend noemen we een **frame**. In deze template draait het spel aan **30 frames per seconde (30 FPS)**.

In de OOP versie worden voor iedere frame de volgende stappen uitgevoerd:

Frame:

1. `handle_input()` — input lezen en verwerken
2. `update_game_state()` — posities aanpassen, botsingen detecteren
3. `draw_elements()` — alles opnieuw tekenen

Omdat dit elke frame opnieuw gebeurt, lijkt het alsof de objecten vloeiend over het scherm bewegen.

1. Speler laten bewegen

De speler tonen

De tekenfase van het spel zit in de `draw_elements()` methode van de `Game`-klasse. Alles wat je daarin tekent, verschijnt op het scherm. Het werkt als **een echt canvas**: het eerste dat je tekent, ligt achter de rest. Teken de achtergrond dus altijd als eerste.

Om iets op het scherm te tekenen heb je **3 opties**: `rectangle`, `circle` en `image`. Het aanroepen gaat als volgt:

```
rectangle(x, y, width, height, color)
circle(x, y, radius, color)
image(image_name, x, y, width, height)
```

Voor de `rectangle` en `circle` heb je ook een `color` nodig. Zo kan je bijvoorbeeld `colors.LIGHTGREEN` typen voor een lichtgroene kleur.

Opdracht 1 — in `Game.draw_elements()` (`# TODO: Opdracht 1`): Teken een lichtgroene cirkel linksonder op het scherm. Let erop dat de cirkel volledig in het scherm past!

Tip: Je kan `colors.` typen in je programmeeromgeving om alle mogelijke kleuren te zien. Vergeet niet: `(0, 0)` is **linksonder**.



In plaats van een lichtgroene cirkel laten we wel graag een afbeelding zien als speler. Daarvoor gebruiken we de `image` functie via de `draw()` methode van de `Player`-klasse. Gebruik voor de speler de `"mage.png"` afbeelding.

Opdracht 2 — in `Player.draw()` (`# TODO: Opdracht 2`): Teken de speler met de `"mage.png"` afbeelding op `self.position` met `self.size` als breedte en hoogte. Roep daarna `self.player.draw()` aan in `draw_elements()` en verwijder de tijdelijke cirkel.

Tip: Gebruik `self.position.x` en `self.position.y` als coördinaten voor de afbeelding.

Test checkpoint: Op dit punt zou je de speler als afbeelding moeten zien op het scherm. Test dit door het spel te starten voordat je verdergaat.

De speler bewegen

Nu we de speler zien, kunnen we die laten bewegen. We doen dat door in `handle_input()` de `self.direction` in te stellen op basis van de ingedrukte pijltjestoetsen. De `move()` methode — die al ingevuld is — past daarna de positie aan.

Om invoer te lezen gebruik je de `input` functie. Zo kan je opvragen of de linkerpijltjestoets ingedrukt wordt:

```
if input(K_LEFT):
```

Opdracht 3 — in `Player.handle_input()` (`# TODO: Opdracht 3`): Reset `self.direction` elke frame naar `Vector(0, 0)`. Stel daarna de x- en y-component in op basis van de ingedrukte pijltjestoetsen: -1 voor links/omlaag en 1 voor rechts/omhoog. Zorg er ook voor dat de speler niet buiten het scherm kan bewegen.

Tip: Begrens de positie van de speler zodat die niet buiten het scherm kan. De speler mag niet verder links dan 0 en niet verder rechts dan `SCREEN_WIDTH - self.size`. Analoog voor boven en onder.

Test checkpoint: Op dit punt zou je de speler soepel in alle richtingen moeten kunnen bewegen. Controleer ook of de speler binnen het scherm blijft. Laat jullie werk even controleren voordat je verder gaat!

2. Vijanden

Het spel is zeker niet compleet zonder vijanden. Vijanden worden al automatisch gespawnt door `spawn_enemies()` in de gegeven code — jullie hoeven dat niet zelf te implementeren.

De vijanden tonen

Opdracht 4 — in `Enemy.draw()` (`# TODO: Opdracht 4`): Teken de vijand met de `"enemy.png"` afbeelding op `self.position` met `self.size` als breedte en hoogte. Roep daarna `enemy.draw()` aan in `draw_elements()` via een for-loop.

Test checkpoint: Vijanden moeten nu zichtbaar zijn op het scherm. Controleer of ze op verschillende posities verschijnen aan de randen.

De vijanden bewegen

Voor de vijanden te laten bewegen vergelijken we hun positie met die van de speler. Anders dan in de imperatieve versie staat de bewegingslogica nu in de `move()` methode van de `Enemy`-klasse zelf.

Opdracht 5 — in `Enemy.move()` (`# TODO: Opdracht 5`): Beweeg de vijand stap voor stap naar de speler toe. Vergelijk de x- en y-positie van de vijand met de doorgegeven spelerpositie, en pas de positie van de vijand telkens aan met `self.speed` in de juiste richting.

Tip: Als de vijand links van de speler staat, moet hij naar rechts bewegen, en omgekeerd. Analoog voor boven en onder. Vergelijk `self.position.x` met `player_x` en doe hetzelfde voor y.

Test checkpoint: Vijanden moeten nu vloeiend naar de speler toe bewegen. Probeer te ontwijken en controleer of ze je volgen!

3. Kogels

De game is zo bijna compleet, maar een bullet-hell zonder bullets is niet veel fun. Dit is ook het moeilijkste deel, dus aarzel niet om vragen te stellen!

De kogels tonen

Opdracht 6 — in `Bullet.draw()` (`# TODO: Opdracht 6`): Teken de kogel met de `"bullet.png"` afbeelding op `self.position` met `self.size` als breedte en hoogte.

Kogels schieten en cooldown

Wanneer de speler beweegt, moet er een kogel afgevuurd worden in die richting. We voegen een nieuwe `Bullet` toe aan `self.bullets` wanneer de speler beweegt én de cooldown op 0 staat.

De **cooldown** is een teller die na elk schot op een bepaalde waarde wordt gezet en elke frame met 1 verlaagd wordt. Je mag pas opnieuw schieten als de cooldown 0 is. De afname van de cooldown is al in de gegeven code voorzien.

Opdracht 7 — in `Game.handle_input()` (`# TODO: Opdracht 7`): Controleer of de cooldown 0 is en of de speler een richting heeft (minstens één component van `self.player.direction` is niet 0). Maak dan een nieuwe `Bullet` aan op de positie van de speler met de richting van de speler. Voeg hem toe aan `self.bullets` en stel de cooldown in op een waarde naar keuze (bijv. 20).

Tip: Een `Bullet` aanmaken doe je met `Bullet(x, y, size, speed, direction)`. Gebruik de waarden uit `self.bullet_speed` en `self.bullet_size` die al in de `Game`-klasse staan.

De volledige tekenfase

Opdracht 8 — in `Game.draw_elements()` (`# TODO: Opdracht 8`): Vul de tekenfase volledig in. Teken achtereenvolgens:

1. De achtergrond: `image("bg.png", 0, 0, SCREEN_WIDTH, SCREEN_HEIGHT)`
2. Alle vijanden via `enemy.draw()`
3. Alle kogels via `bullet.draw()`
4. De speler via `self.player.draw()`
5. Score en highscore als tekst linksbovenaan

Test checkpoint: Het volledige spel moet nu werken. De speler beweegt en schiet, vijanden bewegen naar de speler, en het spel eindigt bij aanraking. Laat alles controleren voordat je verdergaat!

4. Eindresultaat

Als je alle stappen hebt gevolgd heb je nu een volledig functioneel spel, goed gedaan! Laat zeker alles even controleren voordat je verder gaat met uitbreidingen!

In de volgende fase zullen jullie het spel mogen uitbreiden met jullie eigen ideeën. Om jullie toch wat op weg te helpen zijn hier nog enkele voorstellen:

- Pas de code aan zodat de speler altijd naar de positie van de muiscursor schiet in plaats van in de looprichting.
- Laat vijanden soms een item vallen als ze sterven. Als de speler dit oppakt, krijgt hij een bonus:
 - **Machinegeweer:** Verlaag de cooldown tijdelijk zodat je supersnel kunt schieten.
 - **Shotgun:** Schiet 3 kogels tegelijk in een waaivorm.
 - **Speed boost:** Verhoog tijdelijk `self.player.speed`.
- Maak variaties van vijanden door een subklasse van `Enemy` te maken, zoals een "Tank" die langzaam is maar meerdere kogels nodig heeft.
- Laat vijanden sneller worden naarmate de score oploopt.

Het belangrijkste blijft dat jullie je eigen ideeën tot leven brengen, dus laat je niet beperken door deze voorstellen!